



Lab 04 – Worksheet

Name: Mohammad Tahir

ID: mt11830

Section: T3

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Provide appropriately commented codes.

Task01_a:

```
li x10, 2
fact:
addi sp, sp, -8 # adjust stack for 2 items
sw x1, 4(sp) # save return address
sw x10, 0(sp) # save argument n

addi x5, x10, -1 # x5 = n-1
bge x5, x0, L1 # if (n-1) >= 0, go to L1
addi x10, x0, 1 # return 1
addi sp, sp, 8 # pop stack
jalr x0, 0(x1) # return

L1:
addi x10, x10, -1 # argument = n-1
jal x1, fact # recursive call

addi x6, x10, 0 # save result of fact(n-1)
lw x10, 0(sp) # restore original n
lw x1, 4(sp) # restore return address
addi sp, sp, 8 # pop stack

mul x10, x10, x6 # n * fact(n-1)
jalr x0, 0(x1)

Exit:
```



Task01_b:

```
main:
    li    x10, 4          # Set input n = 4
    jal   x1, fact_iter    # Call factorial function
    li    x17, 10         # Load exit code
    ecall                          # Stop the simulator

fact_iter:
    li    x5, 1           # acc = 1

loop:
    bge   x0, x10, done    # If 0 >= n (n <= 0), exit loop
    mul   x5, x5, x10      # acc = acc * n
    addi  x10, x10, -1     # n = n - 1
    j     loop            # Repeat loop

done:
    mv     x10, x5         # Put final answer in x10
    jalr   x0, 0(x1)       # Return to main
```

**Add snip of relevant Registers / Memory locations where you think results are obtained. Make sure the irrelevant area of snip is cropped. Make sure the irrelevant area of snip is cropped.*

--



```
✓ PC
  PC = 8
> PRIV
> CSR
✓ Integer
  x00 (zero) = 0
  x01 (ra) = 8
  x02 (sp) = 2147483632
  x03 (gp) = 268435456
  x04 (tp) = 0
  x05 (t0) = 24
  x06 (t1) = 0
  x07 (t2) = 0
  x08 (s0) = 0
  x09 (s1) = 0
  x10 (a0) = 24
  x11 (a1) = 0
  x12 (a2) = 0
  x13 (a3) = 0
  x14 (a4) = 0
  x15 (a5) = 0
  x16 (a6) = 0
  x17 (a7) = 0
  x18 (s2) = 0
  x19 (s3) = 0
  x20 (s4) = 0
  x21 (s5) = 0
  x22 (s6) = 0
  x23 (s7) = 0
  x24 (s8) = 0
  x25 (s9) = 0
  x26 (s10) = 0
  x27 (s11) = 0
  x28 (t3) = 0
```

Comments

The fact_iter approach computes the result using a loop that multiplies the current n value into an accumulator register (x5) and decrements n (x10) until it equals zero. The iterative implementation does not require a stack frame because it operates purely as a leaf procedure. Since it does not invoke any other procedures, there is no risk of its return address (x1) or current argument variables being overwritten, allowing it to execute entirely within the CPU's temporary and argument registers without touching memory. In contrast, the recursive implementation in part (a) is a non-leaf procedure that must preserve state across nested calls. It populates the stack by subtracting 8 from the stack pointer (sp) on every single invocation to safely store both the return address (x1) and the argument (x10).

Task 2

Provide appropriately commented codes

```
main:
    addi x10, x0, 4      # Set input argument (num = 4)
    jal x1, ntri         # Call the recursive function

    addi a7, x0, 1       # System call code 1: Print integer
    ecall               # Prints the final answer to the console
```



```
    addi a7, x0, 10    # System call code 10: Exit
    ecall

ntri:
    addi x5, x0, 1     # Load 1 into x5
    ble x10, x5, base  # If num (x10) <= 1, jump to 'base'

    addi sp, sp, -8    # Open 8 bytes of space on the stack
    sw x1, 4(sp)       # Save the return address
    sw x10, 0(sp)      # Save the current 'num'

    addi x10, x10, -1  # Change argument to num - 1
    jal x1, ntri       # Jump back to the top of ntri

    lw x6, 0(sp)       # Load the original 'num' into x6
    lw x1, 4(sp)       # Restore the return address
    addi sp, sp, 8     # Close the stack space

    add x10, x6, x10   # x10 = original num (x6) + returned result
(x10)
    jalr x0, 0(x1)     # Return to the caller

base:
    addi x10, x0, 1    # Set return value to 1
    jalr x0, 0(x1)     # Return to the caller
```

Add screenshot of your results



```
✓ PC
  PC = 12
> PRIV
> CSR
✓ Integer
  x00 (zero) = 0
  x01 (ra)   = 8
  x02 (sp)   = 2147483632
  x03 (gp)   = 268435456
  x04 (tp)   = 0
  x05 (t0)   = 1
  x06 (t1)   = 4
  x07 (t2)   = 0
  x08 (s0)   = 0
  x09 (s1)   = 0
  x10 (a0)   = 10
  x11 (a1)   = 0
  x12 (a2)   = 0
  x13 (a3)   = 0
  x14 (a4)   = 0
  x15 (a5)   = 0
  x16 (a6)   = 0
  x17 (a7)   = 1
  x18 (s2)   = 0
  x19 (s3)   = 0
```

Comments

The ntri procedure is a non-leaf recursive function, so it must preserve its state across nested calls. The stack gets populated on every recursive invocation by decrementing the stack pointer (sp) by 8 bytes to allocate space, then storing the return address (x1) and the caller's current argument (x10).

Task 3

Provide appropriately commented codes

```
.data
```



```
my_array: .word 3, 1, 4, 1, 5, 9
len: .word 6

.text
.globl main
main:
    la x10, my_array      # Load base address of array into x10
    lw x11, len           # Load length of array into x11
    jal x1, bubble        # Call bubble sort function

    li a7, 10             # System call for exit
    ecall                 # Stop simulator

bubble:
    beq x10, x0, ret_b    # if a == NULL, return
    beq x11, x0, ret_b    # if len == 0, return
    addi x5, x0, 0        # i = 0

outer:
    bge x5, x11, ret_b    # if i >= len, end sorting
    addi x6, x5, 0        # j = i

inner:
    bge x6, x11, next_i   # if j >= len, loop to next i

    slli x7, x5, 2        # Calculate i * 4 (byte offset)
    add x7, x10, x7       # Address of a[i]
    lw x28, 0(x7)         # Load a[i] into x28

    slli x29, x6, 2       # Calculate j * 4 (byte offset)
    add x29, x10, x29     # Address of a[j]
    lw x30, 0(x29)        # Load a[j] into x30

    bge x28, x30, dont_swap # if a[i] >= a[j], skip the swap

    sw x30, 0(x7)         # Store a[j] into a[i]'s address
    sw x28, 0(x29)        # Store original a[i] into a[j]'s address

dont_swap:
    addi x6, x6, 1        # j++
    jal x0, inner         # Repeat inner loop

next_i:
    addi x5, x5, 1        # i++
    jal x0, outer         # Repeat outer loop
```



```
ret_b:
    jalr x0, 0(x1)      # Return to main caller
```

Add screenshots of your results

✓ Integer

x00 (zero)	= 0
x01 (ra)	= 20
x02 (sp)	= 2147483632
x03 (gp)	= 268435456
x04 (tp)	= 0
x05 (t0)	= 0
x06 (t1)	= 6
x07 (t2)	= 6
x08 (s0)	= 0
x09 (s1)	= 0
x10 (a0)	= 268435456
x11 (a1)	= 6
x12 (a2)	= 0
x13 (a3)	= 0
x14 (a4)	= 0
x15 (a5)	= 0
x16 (a6)	= 0
x17 (a7)	= 10

Address	+0	+1	+2	+3
0x10000018	06	00	00	00
0x10000014	09	00	00	00
0x10000010	05	00	00	00
0x1000000C	01	00	00	00
0x10000008	04	00	00	00
0x10000004	01	00	00	00
0x10000000	03	00	00	00
0x0FFFFFFC	00	00	00	00
0x0FFFFFF8	00	00	00	00
0x0FFFFFF4	00	00	00	00
0x0FFFFFF0	00	00	00	00

**Comments:**

This procedure successfully translates the nested loops of the C bubble sort implementation into RISC-V assembly. Because bubble is a leaf procedure that does not call any other functions, it does not require a stack frame; it executes efficiently using only temporary registers (x28–x31) for memory operations. The byte addresses for the array elements are calculated dynamically.

Task 4

Provide appropriately commented codes

```
# Recursive Array Sum: Combines array memory access (Task 3),
recursion & stack (Task 2), and addition (Task 1)

.data
my_array: .word 5, 10, 15    # Create an array with 3 numbers in
memory
len: .word 3                 # Set the length of the array to 3
.text
.globl main
main:
    la x10, my_array         # Load the base address of the array
into x10
    lw x11, len               # Load the length of the array into x11
    jal x1, array_sum         # Jump and link to the recursive
function

    addi a7, x0, 1            # Set system call code to 1 (Print
Integer)
    ecall                    # Print the final computed sum to the
console

    addi a7, x0, 10           # Set system call code to 10 (Exit
Program)
    ecall                    # Stop the simulator

array_sum:
    beq x11, x0, base_case    # If length is 0, jump to the base case

    addi sp, sp, -8           # Open 8 bytes of space on the stack
    sw x1, 4(sp)              # Save the return address on the stack
    lw x7, 0(x10)             # Load the current array element into x7
    sw x7, 0(sp)              # Save the current element on the stack
```



```
    addi x10, x10, 4      # Move the array pointer forward by 4
bytes (next element)
    addi x11, x11, -1     # Decrease the remaining length by 1
    jal x1, array_sum     # Recursively call the function for the
rest of the array

    lw x7, 0(sp)          # Restore the saved array element from
the stack
    lw x1, 4(sp)          # Restore the saved return address from
the stack
    addi sp, sp, 8        # Close the 8 bytes of space on the
stack

    add x10, x10, x7      # Add the current element to the total
sum returned
    jalr x0, 0(x1)        # Return to the caller

base_case:
    addi x10, x0, 0       # Set the return value (sum) to 0
    jalr x0, 0(x1)        # Return to the caller
```

Add screenshots of your results

```
✓ PC
  PC = 24
> PRIV
> CSR
✓ Integer
  x00 (zero) = 0
  x01 (ra)   = 20
  x02 (sp)   = 2147483632
  x03 (gp)   = 268435456
  x04 (tp)   = 0
  x05 (t0)   = 0
  x06 (t1)   = 0
  x07 (t2)   = 5
  x08 (s0)   = 0
  x09 (s1)   = 0
  x10 (a0)   = 30
  x11 (a1)   = 0
  x12 (a2)   = 0
  x13 (a3)   = 0
  x14 (a4)   = 0
  x15 (a5)   = 0
  x16 (a6)   = 0
  x17 (a7)   = 1
  x18 (s2)   = 0
  x19 (s3)   = 0
  x20 (s4)   = 0
```

**Comments:**

This RISC-V assembly program calculates the total sum of an integer array by utilizing a recursive function. It begins by initializing a memory array and passing its base address and length into the main procedure. Each recursive iteration advances the memory pointer by four bytes to read the next consecutive array element. The function maintains strict stack discipline by allocating eight-byte frames to safely preserve both the return address and the current integer. Upon reaching the zero-length base case, the stack collapses and sequentially adds all saved elements to produce the final computed output.



Lab 04 – Nest Procedures and Sorting

Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	AR 7 Report Submission & Github
Task 1	/10	/05	/10 & /10
Task 2	/10	/10	
Task 3	/10	/10	
Task 4	/15	/10	
Total Points	/100 Points		
CLO Mapped	CLO 2		

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.



			Report is submitted after due date.		
--	--	--	--	--	--